

Question 5

Question 5 — what changes at scale, and what is not solved yet.

The question

What changes at scale?

My answer

Scaling this is not copying the current model four hundred times. The engineering problem genuinely changes, and I would rather describe what breaks than claim it simply grows.

First, measurement has to become real. Right now nothing in the system is measured. Two sensors change that: evaporator steam flow and reject conductivity. Those two readings turn the two most load-bearing coefficients from published values into measured ones. They also need calibration and health monitoring, because a drifting sensor is worse than no sensor. The rule I have already built is that if a reading is missing, frozen or implausible, the system holds and credits zero rather than guessing. That rule has to survive scale, not be relaxed for throughput.

Second, the process varies more than one model does. Salt concentration changes with shade depth and dye chemistry. Membrane behaviour changes as membranes age and foul. I currently model the concentration ceiling as a single figure, 60,000 mg/L. A pilot has to test whether that holds across a real operating range or whether it needs to become a function of membrane condition.

Third — and this is the largest known gap — the treatment plant is shared. My optimiser handles one machine's queue. A real mill runs many machines feeding one membrane system and one evaporator, and a CETP serves hundreds of units feeding one shared plant. That is a genuinely different optimisation: machines compete for shared treatment capacity, and the best plan for one machine is not the best plan for the plant. I have not solved that, and I will not pretend otherwise. It is the specific scale-up problem, not a detail.

There is a fourth thing that is easy to miss. The scheduling decision is daily, but the decision that moves the energy bill most — switching to low-salt chemistry — is a procurement and quality decision on a much slower cycle. At scale, the system has to serve both clocks: a daily recommendation to the planner, and a periodic case to the people who buy chemicals and approve recipes.

Operationally, the planner uses it during scheduling, the quality supervisor keeps authority over wash-off release, the unit owner receives the financial benefit, and the CETP is the most efficient commercial channel, because one connection can serve many member units and its incentive already points the same way — less salt arriving means less steam bought and less salt to store.

My rollout is staged over about 32 weeks: baseline metering first, then shadow mode where recommendations are generated but change nothing, then advisory use with human approval, then a controlled wash-off release trial, then the chemistry decision, then expansion to more units. A quality failure — a wash-fastness failure in particular — stops that stage rather than being absorbed to protect a target.

The point of that sequence is not to prove I was right. It is to find out whether the decision logic survives real variability, shared equipment and real constraints. If it does, the same architecture extends from one machine toward plant and cluster-level optimisation. If it does not, I would rather know in week six than

after a cluster has paid for it.

Supporting notes

Why I answered it this way

Scale questions are usually answered with ambition. I answered with the three specific things that break, because naming your own largest gap is more convincing than claiming you do not have one — and because the shared-treatment gap is real and a process engineer on the panel will spot it immediately if I hide it.

The staged rollout is in there to show that I understand adoption risk, not just technical risk. Shadow mode in particular matters: it is how you earn the right to influence a factory's decisions, by being correct for four weeks while changing nothing.

Claims and where they rest

Claim	Basis
Nothing currently measured	<code>core/factors.py</code> : 0 coefficients classed MEASURED, enforced by a test
Two readings unlock the most	Sensitivity sweep: steam cost and reject ceiling dominate the result
Fail-closed on bad sensor data	<code>core/telemetry.py</code> ; <code>automatic_release</code> hardcoded <code>False</code> ; 5 fault modes tested
Missing data credits zero	<code>tests/test_engine.py</code> safety-interlock tests
Concentration ceiling is a single modelled figure	<code>ro_max_reject_tds_mg_l</code> = 60,000 mg/L, inside a published 15,000–80,000 band
Optimiser handles one machine	<code>core/optimizer.py</code> ; exhaustive to 8 lots, heuristic beyond
CETP incentive aligns	Less salt means less steam and less stored salt; over 1 lakh tonnes currently stored with no disposal route
32-week staged rollout	<code>/api/pilot</code> ; phases are defined in the engine, not invented for this answer

What I deliberately did not claim

- I did not claim the shared-treatment optimisation is solved, or nearly solved.
- I did not claim the 60,000 mg/L ceiling holds universally.
- I did not promise a cluster rollout timeline. The 32 weeks covers one unit through to first expansion, not a cluster.

The hardest follow-up, and my answer

"You have solved the easy version. The real problem is the shared plant."

That is correct, and it is why I put it in the answer rather than waiting to be asked. What I have built is the decision layer and the physics that drives it, proven on a single machine, with the accounting and the safety behaviour already in place. The shared-resource optimisation sits on top of the same consequence

model — the thing it needs to know, that reject volume is set by salt, does not change when the plant is shared. That is why I started where I did rather than starting with the harder scheduling problem and an unexamined cost model underneath it.